

Longest Substring with K Distinct Characters

Two approaches, visualized execution, and the core logic to remember

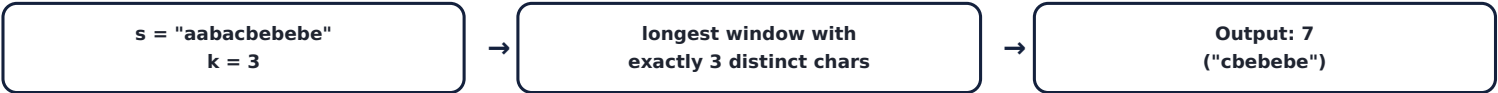
Brute Force · $O(n^2)$

Sliding Window · $O(n)$

[Reference notes](#) · [LongestSubstringWithKUnique.java](#)

Problem Statement

Given a string `s` of lowercase letters and an integer `k`, return the length of the longest substring containing **exactly** `k` distinct characters. Return `-1` if no such substring exists.



Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	$O(n^2)$	$O(k)$	Restart a fresh map at every start index, stop early past k distinct
Sliding Window	$O(n)$	$O(k)$	Expand right, shrink left when $> k$ distinct, record when exactly k

Big picture: A frequency map doubles as a distinct-character counter — `map.size()` is the number of distinct characters in the window. The sliding window keeps that map up to date incrementally instead of rebuilding it for every start index.

Heads-up: the code uses `java.util.Map` and `java.util.HashMap` — make sure `import java.util.HashMap;` and `import java.util.Map;` are present in the source file.

1 · Brute Force

$O(n^2)$ time

$O(k)$ space

For each
start index i



Extend j ,
count chars in map



size == k ?
record length



size > k ?
break

```
// try every start, extend until too many distinct chars, then stop
private static int bruteForce(String s, int k) {
    int maxLength = -1;

    for (int i = 0; i < s.length(); i++) {
        Map<Character, Integer> map = new HashMap<>();

        for (int j = i; j < s.length(); j++) {
            char c = s.charAt(j);
            map.put(c, map.getOrDefault(c, 0) + 1);

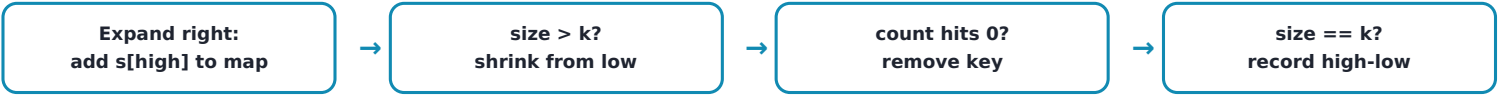
            if (map.size() == k) {
                maxLength = Math.max(maxLength, j - i + 1);
            }

            if (map.size() > k) {
                break;
            }
        }
    }
    return maxLength;
}
```

Recall trick: "Try every start, extend until too many distinct chars, then stop." Correct, but rebuilds the map from scratch for every i — $O(n^2)$ in the worst case.

2 · Sliding Window (Variable Size)

O(n) time O(k) space



```
// expand right; too many distinct? shrink left; record only when exactly k
private static int usingSlidingWindow(String s, int k) {
    int high = 0;
    int low = 0;
    int maxLength = -1;
    Map<String, Integer> map = new HashMap<>();

    String[] stringArr = s.split("");

    while (high < stringArr.length) {
        map.put(stringArr[high], map.getDefault(stringArr[high], 0) + 1);
        high++;

        while (map.size() > k) {
            map.put(stringArr[low], map.getDefault(stringArr[low], 0) - 1);
            if (map.get(stringArr[low]) == 0) {
                map.remove(stringArr[low]);
            }
            low++;
        }

        if (map.size() == k) {
            maxLength = Math.max(maxLength, high - low);
        }
    }
    return maxLength;
}
```

Trace for s = "aabacbebe", k = 3:

high (after++)	char added	map after shrink	low	action	maxLength
1-4	a, a, b, a	{a:3, b:1}	0	size 2 < 3 → skip	-1
5	c	{a:3, b:1, c:1}	0	size 3 → record 5	5
6	b	{a:3, b:2, c:1}	0	size 3 → record 6	6
7	e	{b:1, c:1, e:1}	0→4	size 4 > 3 → shrink (drops a,a,b,a) → record 3	6
8	b	{b:2, c:1, e:1}	4	record 4	6
9	e	{b:2, c:1, e:2}	4	record 5	6
10	b	{b:3, c:1, e:2}	4	record 6	6
11	e	{b:3, c:1, e:3}	4	record 7 ("cbebebe")	7

Recall trick: "Expand right; if more than k distinct, shrink left until it's k again; record only when it's exactly k." The key detail: **remove the key when its count hits 0** — `map.size()` is the distinct count, so a zero-count key left behind would break the logic. Adding one char raises the distinct count by at most 1, so shrinking lands on exactly k, never past it.

TL;DR — For Future Me

- **Best solution:** variable-size sliding window + frequency map, O(n) — expand right, shrink left when > k distinct, record only when exactly k.
- **Brute force** is O(n²) — rebuilds the map for every start.
- **Must remember:** remove a key when its count reaches 0 — `map.size()` is the distinct-character count.
- **Pattern:** the "longest window with at most/exactly k distinct" template — also appears in *Longest Substring Without Repeating Characters*, *Fruit Into Baskets*, and *Longest Substring with At Most K Distinct Characters*.